

Projeto de Programação Multiparadigma [v1.0] 2022/2023

Enunciado

Um projeto de programação multiparadigma tira partido das vantagens de diferentes paradigmas de programação de forma a produzir uma solução mais adequada (p.e. desempenho, leitura/interpretação, manutenção). Uma divisão clássica consiste na utilização do paradigma de programação orientado aos eventos para o desenvolvimento da **camada de apresentação** (*User Interface*) e outro(s) paradigma(s) (funcional, orientado aos objetos, imperativo, lógico) para o desenvolvimento da **camada de negócio**. A **camada de dados** completa uma típica arquitetura de 3 camadas.

Aviso: O trabalho entregue deve corresponder ao esforço individual de cada grupo, e em nenhum caso deve ser copiado código que será entregue (incluindo proveniente de plataformas de Inteligência Artificial p.e., *ChatGPT*, *CoPilot*, etc.). A deteção de código copiado será realizada por software especializado bastante sofisticado. Casos de plágio óbvio serão penalizados com a anulação do projeto, o que implica a reprovação à Unidade Curricular (UC). Adicionalmente, a situação será reportada à Comissão Pedagógica da ISTA/Conselho Pedagógico do ISCTE-IUL. Serão penalizados da mesma forma tanto os alunos que fornecem código (se for o caso) como os que copiam código.

Grupos de Trabalho

O trabalho deverá ser realizado por grupos de **3 elementos** (exceções necessitam de aceitação explícita por parte do coordenador da UC).

Datas

- Entrega nº1: até 17 de abril (via Moodle) contendo a 1ª parte (no mínimo com as tarefas T1, T2, T3, T5 e T6)
- Discussão nº1: 26 a 28 de abril (no horário habitual das aulas)
- Entrega nº2: até 22 de maio (via Moodle) contendo a 1ª parte (melhorada) e a 2ª parte
- Discussão nº2: durante uma das aulas (podem ser necessárias *slots* adicionais) da última semana de aulas – 22 a 26 de maio

Introdução

O objetivo deste projeto é o desenvolvimento de competências de programação que permitam desenvolver, em colaboração, aplicações multiparadigma interativas de média escala. O projeto divide-se em duas partes. A primeira parte corresponde ao desenvolvimento da camada de negócio utilizando, obrigatoriamente, a linguagem de programação **Scala** seguindo os princípios de programação funcional pura (exceções serão explicitamente indicadas). A segunda parte corresponde ao desenvolvimento da camada de apresentação (interativa) que deverá ser desenvolvida utilizando **JavaFX**.

Parte 1: Camada de Negócio

Objetivos: Desenvolver um conjunto de métodos, utilizando o paradigma de programação funcional, capazes de garantir os objetivos funcionais da solução proposta.

Parte 2: Camada de Apresentação Interativa

Objetivos: Desenvolver uma *text-based User Interface* (TUI) e uma *Graphical User Interface* (GUI) que permitam uma interação fácil e intuitiva com a aplicação.

Requisitos

O projeto deverá, adicionalmente, satisfazer os seguintes requisitos:

Funcionais

1. Permitir executar com sucesso as várias tarefas/métodos (ver secção Tarefas);
2. Manter estado entre execuções, i.e., guardar o estado de jogos não terminados para execuções futuras.

Não funcionais

1. Apresentar ambas interfaces: gráfica (GUI) e textual (TUI);
2. Os utilizadores deverão conseguir interagir corretamente (com o menor número de erros cometidos) e com facilidade (equilíbrio adequado entre quantidade de informação apresentada e número de ações necessárias para realizar uma tarefa) sem necessidade de treino.

Tema – Jogo de tabuleiro *Hex*

Neste trabalho prático ocupar-nos-emos com a implementação do jogo de tabuleiro Hex¹

Tarefas

- **T1:** implementar o método `randomMove(board: Board, rand: MyRandom): ((Int, Int), MyRandom)` responsável por gerar uma coordenada aleatória válida para a próxima jogada (recebe e devolve `MyRandom` para poder ser puro);
- **T2:** implementar o método `play(board: Board, player: Cells.Cell, row: Int, col: Int): Board` responsável por efetuar uma dada jogada (i.e., do computador ou do jogador);
- **T3:** representar, visualmente, as jogadas no tabuleiro;
- **T4:** implementar o método `hasContiguousLine` responsável por verificar se o computador ou o jogador ganhou o jogo;
- **T5:** permitir que seja possível após cada jogada realizar undo i.e., anular a última movimentação do jogador e do computador;
- **T6:** desenvolver uma TUI disponibilizando as várias opções de interação com o jogo (tabuleiro de dimensão configurável);
- **T7:** desenvolver uma GUI (tabuleiro de dimensão fixa: 5x5) com as mesmas opções que a TUI e, adicionalmente, configurações do jogo (i.e., quem inicia, nível de dificuldade, escolha de cores).

Fatores de valorização:

- Adicionar um nível de dificuldade intermédio ao jogo melhorando o desempenho do computador através da seleção de posições livres conexas;
- Utilização de conceitos de programação funcional “avançada” (p.e., *fold*, *tail recursion*);

Tipo de Dados a Utilizar Obrigatoriamente

```
type Board = List[List[Cells.cell]]

object Cells extends Enumeration {
  type Cell = Value
  val Red, Blue, Empty = Value
  ...
}
```

¹ [https://en.wikipedia.org/wiki/Hex_\(board_game\)](https://en.wikipedia.org/wiki/Hex_(board_game))

Exemplos de Execução do Jogo *Hex*

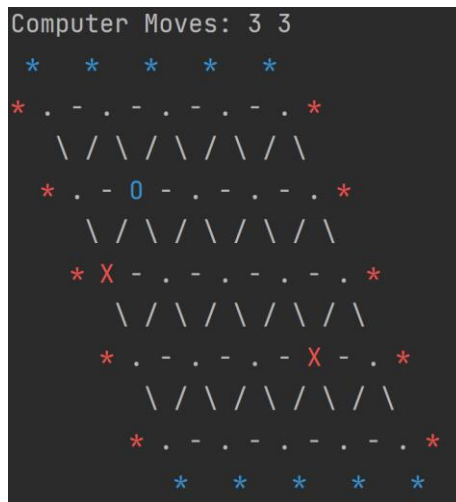


Figura 1 – Exemplo de execução parcial do *Hex Game* (5x5) via TUI

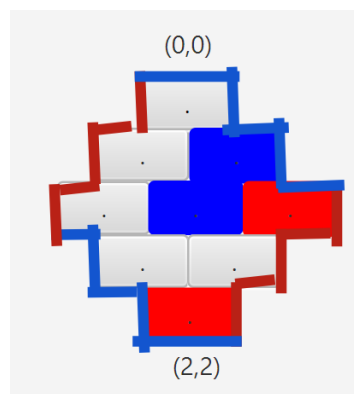


Figura 2 - Exemplo de execução parcial do *Hex Game* (3x3) via GUI

Avaliação

A realização do projeto é obrigatória para obter aprovação à UC. Não existe qualquer possibilidade de obter aprovação à UC sem realizar o projeto. O projeto é composto por duas partes e realizado em grupo, no entanto, as discussões e aferições são individuais. As classificações possíveis na aferição individual são A, B ou C, definindo a nota final individual do projeto (*nf*):

- A: *nf* igual à classificação do projeto;
- B: *nf* igual a 80% da classificação do projeto;
- C: reprovação à UC.

A **primeira parte** do projeto (com peso de **75%**) será inicialmente avaliado em termos funcionais (i.e., se os métodos produzem os resultados esperados) e se cumpre os requisitos funcionais (para ser usado numa primeira fase via TUI). A solução desenvolvida deverá, obrigatoriamente, aplicar as seguintes matérias de programação funcional introduzidos na UC:

- Recursividade (no lugar de iteratividade) e *Pattern matching*;
- Imutabilidade (exceto na camada interativa);
- *Pure functions* (exceto na camada interativa);
- Função de ordem superior;
- *Functional Object-Oriented programming*.

Ainda que a funcionalidade exigida tenha sido concretizada, a não aplicação ou aplicação inadequada das matérias mencionadas poderão diminuir a classificação. A avaliação também poderá ser penalizada em função da reduzida qualidade do código (p.e., difícil manutenção, baixa eficiência, dificuldade de interpretação, duplicação, complexidade desnecessária).

A **segunda parte** do projeto (com peso de **25%**) será avaliada em termos da qualidade e usabilidade das interfaces (TUI e GUI) desenvolvidas, do cumprimento dos requisitos e da correta aplicação da matéria sobre engenharia de sistemas interativos (*Event-driven Programming*, *UI patterns* e *Tree Testing*). Apesar de ser muito simples, a arquitetura de informação da GUI deve ser avaliada via *Tree Testing* com 2 tarefas e 3 participantes².

Apesar de não ter um peso na avaliação a utilização de tecnologias para suporte à programação em equipa (e.g., Git, GitHub) é aconselhado.

Os alunos poderão obter feedback juntos dos professores das respetivas turmas sobre o progresso do projeto, e deverão seguir as recomendações dadas.

Os projetos são classificados de acordo com os seguintes pesos:

1ª parte (75%):

- Requisitos funcionais – 40%
- Fatores de valorização – 10%
- Correta e completa aplicação da matéria de Programação Funcional – 20%
- Qualidade do código – 5%

2ª parte (25%):

- Qualidade das interfaces desenvolvidas – 15%
- Correta aplicação da matéria de sistemas interativos – 10%

² p.e., os elementos de cada grupo podem ser participantes no *Tree Testing* de outro grupo

Entrega

Submissões: ambas as entregas deverão ser, obrigatoriamente, realizadas por cada grupo através do Moodle.

Dados e formatação:

- Projeto com todo o código fonte desenvolvido em formato *Zip* ("File/Export/Project to Zip File...") **contendo todos os dados necessários para poder testá-lo**. O nome do projeto deve incluir o **número do grupo e nome dos vários membros** (*NúmeroGrupo_Nome1_Nome2_Nome3*).
Sugestão: verifiquem que a importação do vosso projeto numa máquina diferente ocorre com sucesso antes de efetuarem a submissão;
- Resultados do *Tree Testing* (ficheiro: *Link_TreeTesting.txt*) contendo o link para os resultados obtidos (optimalworkshop link);
- Possíveis comentários (ficheiro: *Readme.txt*).

Os ficheiros (*Link_TreeTesting.txt*, *Readme.txt*) devem ser adicionados à raiz do projeto.

Cada membro do grupo deve submeter, individualmente, na entrega final a autoavaliação do trabalho em grupo (ficheiro: *grupo_NúmeroGrupo.pdf*) – ver tabela no anexo I. Este ficheiro deverá ser submetido através do Moodle num outro link de submissão que será criado para o efeito.

Discussão: a discussão é individual e todos os alunos terão de demonstrar ser capazes de fazer um trabalho com o nível de qualidade igual ao que entregaram. Ao entregar o trabalho, os alunos implicitamente afirmam que são os únicos responsáveis pelo código entregue e que todos os membros do grupo participaram de forma equilibrada na sua execução, tendo todos adquirido os conhecimentos necessários para produzir, individualmente, um trabalho equivalente.

Anexo I - Avaliação do trabalho em grupo

Classifique os membros do seu grupo (incluindo-o a si próprio) acerca dos itens seguintes numa escala até 20. Coloque o nome de cada membro no cabeçalho da respetiva coluna.

Responsabilidades	<i>Minha</i> (preencher)	<i>MembroA</i> (preencher)	<i>MembroB</i> (preencher)
Contribuiu para a sua parte do trabalho			
Assistiu a todas as reuniões calendarizadas			

Foi cooperativo			
Teve uma atitude positiva			
Foi tolerante a outros pontos de vista			
Teve espírito de grupo			
Completoou todo o trabalho a si atribuído e no tempo estipulado			
Esforçou-se para produzir resultados de qualidade			
Incluiu os outros membros nas decisões tomadas e discussões			
Ajudou os outros membros do grupo			